Ground Station Network

Requirements and communication protocol
Design revision 0.1 / proposed wire protocol 1.0

MIT Rocket Team
Avionics and GNC

25 October 2026

Document status and decision record

Design specification; not an implementation report This document defines requirements for a future station network. It does not claim that the released v0a application implements these services or has passed the tests below.

The agreed deployment comprises ten computers on one reachable local network: Launch Telemetry, Launch Video, and Telemetry/Video computers at each of four Away stations. The user has accepted the embedded-service architecture and H.264 video. This revision makes the proposed network protocol precise enough to guide implementation and interoperability tests. Transport defaults, rates and resource limits below are engineering proposals, to be qualified on the actual network.

Decision	Baseline in this revision
Discovery	Bonjour-compatible mDNS and DNS-SD; one advertised application service per computer. Manual endpoint fallback.
Metadata and snapshots	HTTP API with JSON; current values come from memory, not a repeatedly rewritten text file.
Live telemetry and health	WebSocket event stream, independent of video, with source identity, sequence numbers, bounded resumption and explicit gaps.
Video	H.264/AVC, MPEG transport stream container, SRT live transport over UDP. Preview and main renditions.
Station naming	UI says Launch station. Requested network aliases retain `rt-base-*`; Away aliases use `rt-gs1-*` through `rt-gs4-*`.
Hardware commands	Existing serial protocols remain separate. Unspecified uplink/target-switch/remote-control commands are unsupported in protocol 1.0.
Reference application	The released v0a/0.1.1 manual remains historical evidence of implemented behavior. This specification is a forward design.

Reading convention. SHALL is required to claim conformance to this proposed protocol; SHOULD permits a documented engineering exception; MAY is optional. These words define requirements, not a claim of user approval of every numeric default. All times are milliseconds unless a unit is stated. JSON field names and enum values are case-sensitive. Source listings are illustrative where identified; complete machine-readable examples accompany this report.

Scope. Discovery, identity, mission association, monitoring, raw telemetry provenance, video transport, reconnection, UI behavior, limits and verification. The design runs locally without a cloud service or Internet access. It does not specify a navigation estimator, board firmware, LTU management API, RF modulation, voice communications or launch authorization procedure.

Revision history. Revision 0.1, 25 October 2026: first requirements and protocol draft following the ten-computer network discussion. No application source changes are part of this document.

Contents

Document status and decision record 2

I. System and operator requirements 4

1. Topology, ownership and channel assignment 4

2. Operator-facing requirements 5

3. Data integrity, resource and packaging requirements 6

II. Communication protocol 7

4. Identity, discovery and mission association 7
- 4.1. Identity model . . . 7
- 4.2. DNS-SD records and names . . . 7
- 4.3. Ports and compatibility . . . 8

5. HTTP API and common encoding 8

6. WebSocket lifecycle and event delivery 9
- 6.1. Connection setup and synchronization . . . 9
- 6.2. Message envelope and journal . . . 10
- 6.3. Liveness, clock handling and freshness . . . 11

7. Telemetry observation and status contracts 11
- 7.1. Packet provenance and vehicle attribution . . . 11
- 7.2. Canonical values and physical frames . . . 12
- 7.3. Status and stream health . . . 12

8. H.264 media profile and SRT transport 13
- 8.1. Codec, container and transport are separate choices . . . 13
- 8.2. Renditions and manifests . . . 13
- 8.3. Session allocation, connection and lease . . . 14
- 8.4. Promotion, bandwidth and failure recovery . . . 15
- 8.5. Media timing and recording . . . 15

III. Verification and implementation boundaries 16

9. Performance targets and conformance checks 16

10. Implementation order and open decisions 17

11. Traceability and document maintenance 18

A. Example exchanges 19
- A.1. Telemetry service identity . . . 19
- A.2. Client subscription . . . 19
- A.3. Received observation . . . 20
- A.4. Video session request and response . . . 21
- A.5. Expired resume cursor . . . 21

Part I.
System and operator requirements

1. Topology, ownership and channel assignment

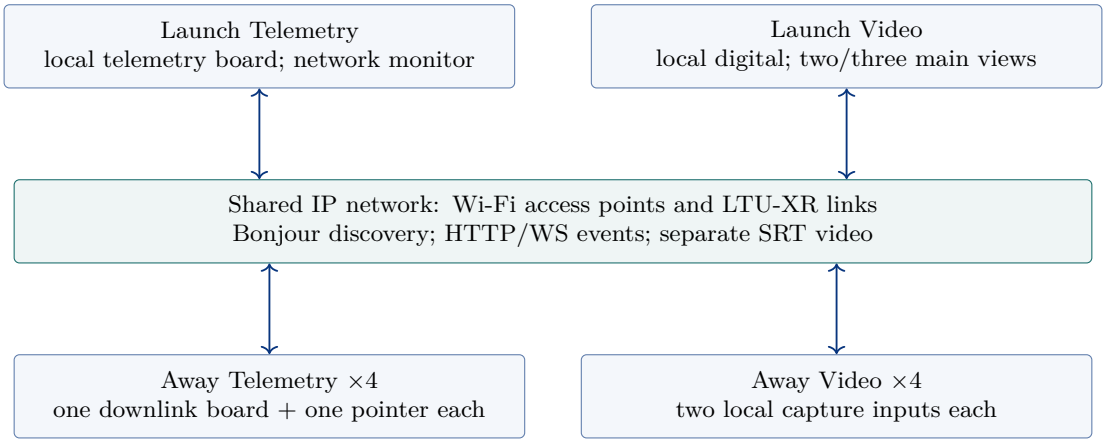


Figure 1: Ten independent application services. The network is shared; device ownership stays local. Arrows indicate network reachability, not command authority.

Logical slot	Service instance / preferred alias stem	Local responsibilities
`launch/telem`	`rt-base-telem`	One telemetry board; local downlink; future software-enabled uplink. No physical pointer. Monitor all eight Away computers and Launch Video.
`launch/video`	`rt-base-video`	One local digital capture input; receive and display all Away video feeds.
`gsN/telem`	`rt-gsN-telem`, $N = 1 \ldots 4$	Own the station's downlink telemetry board and antenna pointer board. Publish their independent status and telemetry observations.
`gsN/video`	`rt-gsN-video`, $N = 1 \ldots 4$	Own the station's USB camera inputs; publish encoded video and capture/stream status.

SYS-01 The application SHALL support all ten slots simultaneously. A station number, role and vehicle mode SHALL come from the wizard or equivalent CLI options. Discovery SHALL not silently change those choices.

SYS-02 Each physical serial device or camera SHALL have one local owner. Network clients subscribe to that owner's data. A second view SHALL reuse the capture pipeline rather than open the same camera again. Duplicate camera assignment and duplicate serial ownership SHALL remain prohibited.

SYS-03 Launch Telemetry SHALL receive remote observations in addition to its own board input. Remote data SHALL retain its source and SHALL not masquerade as a local board connection. Away stations SHALL remain useful when either Launch computer is absent. Launch Video SHALL operate independently of Launch Telemetry.

The network consists logically of one LAN. The physical LTU-XR links, PoE hardware and access points carry IP traffic; they are not telemetry sources by themselves. An LTU-XR bridge and a Wi-Fi access point may be separate devices. The application must not assume that an LTU radio itself implements a client Wi-Fi access point.

Mode	Publishing slot	Local channel IDs	Launch main slots
Balius	Launch Video	`balius.digital`	Digital; Analog
Balius	Away Video 1–4	`balius.digital`, `balius.analog`	Same two logical channels
Iris	Launch Video	`sustainer.digital`	Sustainer Digital; Sustainer Analog; Booster Analog
Iris	Away Video 1–3	`sustainer.digital`, `sustainer.analog`	Matching logical channel
Iris	Away Video 4	`sustainer.digital`, `booster.analog`	Matching logical channel

SYS-04 Iris SHALL expose three system channels while each Away station has two local camera feeds under the latest hardware diagram. Older references to three cameras at every Away station are superseded by this mapping.

SYS-05 Iris target selection SHALL represent desired and confirmed targets separately. Until firmware identifies a packet's vehicle or confirms the board switch, reception alone SHALL not establish its vehicle. The Launch board remains one board, not separate uplink and downlink serial devices.

2. Operator-facing requirements

UI-01 Launch Telemetry SHALL show four station rows with distinct Telemetry and Video computer cells, plus a separate Launch Video companion indicator. Discovery, app heartbeat, hardware connection, RF reception and video freshness SHALL be independently visible without changing tabs.

UI-02 A cell SHALL expose station/role, IP endpoint, connection state, last heartbeat age, protocol compatibility and mission identity. Details SHALL include board state, packet age, observed loss/gaps, RSSI when provided, pointer mode and commanded pose. Measured pose SHALL remain unknown when unavailable.

UI-03 Network health SHALL use text plus color: online/green, stale/amber, offline/red, undiscovered or inapplicable/gray. Conflict and incompatible states SHALL be explicit. Rocket state colors remain the existing independent convention: green GROUND_TESTING, blue PREFLIGHT, red FLIGHT, orange recovery/post-apogee, black other.

UI-04 A connected pointer SHALL not imply a rocket RF link. USB frames SHALL not imply receiver RF lock: a receiver may produce a no-signal screen. Absent RF metrics SHALL display unknown rather than a fabricated quality score.

UI-05 Launch Video SHALL display two large channels for Balius, three for Iris, and all eight Away feeds as station-labeled thumbnails. Double-clicking a thumbnail SHALL replace only its matching large logical channel. The operator SHALL be able to restore local digital or choose another matching source.

UI-06 Source selection SHALL remain visible on the large view. On loss, the last image MAY remain with an unmistakable stale/offline overlay and age. The UI SHALL not silently substitute another source. An unavailable main rendition MAY show that same source's preview with a visible reduced-quality label.

UI-07 Wi-Fi selection SHALL present the intended network and actual OS connection separately. Permission denial, client isolation, blocked multicast and inaccessible endpoints SHALL yield actionable status and manual-IP entry. No public Internet connection SHALL be required for operation.

3. Data integrity, resource and packaging requirements

DATA-01 Every accepted telemetry observation SHALL carry original bytes when available, decoder identity, station/node identity, vehicle attribution basis, mission, source mode, timing and reception metadata. Unknown values SHALL remain unknown. Forwarding SHALL preserve provenance.

DATA-02 The receive archive SHALL preserve observations independently of UI refresh and network delivery. Multiple stations receiving one rocket packet SHALL remain separate reception records but SHALL not count as independent rocket measurements in a future estimator.

DATA-03 LIVE, REPLAY and SIMULATION sources SHALL never silently mix. Non-live network publication SHALL be explicitly enabled and conspicuously labeled. The default subscription accepts LIVE only. This does not require a new user-facing simulation mode in the application.

NET-01 Telemetry/status and video SHALL use separate connections, worker queues and resource budgets. A blocked media receiver SHALL not block serial ingestion, event journaling, heartbeats or local controls.

NET-02 All queues and messages SHALL have finite limits. Event loss SHALL produce an explicit gap or disconnect, never silent success. Video SHALL favor bounded live latency over displaying an ever-growing backlog.

PKG-01 macOS and Windows packages SHALL contain the discovery, HTTP, WebSocket, H.264 and SRT runtime dependencies. No separately installed media server, Python, FFmpeg command, or Bonjour installer SHALL be required by the operator. OS network/camera permissions remain normal platform requirements.

PKG-02 Packaging SHALL verify actual codec, MPEG-TS and SRT availability in the bundled binaries; finding an FFmpeg executable is insufficient. Software encoding SHALL be available when qualified hardware encoding is unavailable. Dependency licenses and notices SHALL accompany distributions.

Part II.
Communication protocol

4. Identity, discovery and mission association

4.1. Identity model

Field	Definition
`node_id`	Persistent UUID for an application installation; independent of hostname, IP and chosen role. Cloned installations must generate distinct IDs.
`boot_id`	Fresh UUID on every service process start. Sequences and monotonic times are meaningful only within this boot.
`station`	Exactly `launch`, `gs1`, `gs2`, `gs3` or `gs4`.
`role`	Exactly `telem` or `video`. Together with station, identifies a logical slot.
`mission_id`	UUID shared deliberately across the ten computers; `null` until assigned. A human-readable mission name is not a unique identity.
`vehicle_mode`	`balius` or `iris`; must agree with the mission.
`protocol`	Object containing integer `major` and `minor`; this draft specifies 1.0. App version is a separate string.

ID-01 Two live nodes claiming the same slot SHALL create a visible conflict. Neither SHALL replace the existing selected node automatically. DNS name conflict resolution does not resolve logical slot conflicts. Concurrent endpoints presenting the same node ID with different live boots SHALL also be flagged as an identity conflict rather than merged.

ID-02 The wizard's remember checkbox MAY persist user choices; permanent node identity is separate. Changing station, role, mission or source mode while serving SHALL end current subscriptions/media sessions and start a new boot identity. Network data from the previous identity SHALL not leak into the new session. No sequence reset is permitted inside one boot.

Mission assignment is explicit: Launch Telemetry creates or opens a mission UUID; operators join it in the other applications or supply it on the CLI. A receiving UI can display a mismatched peer in diagnostics, but SHALL exclude its data from that mission's telemetry/video workspace. If Launch Telemetry is absent, a previously shared mission UUID still works. There is no implicit assignment from an SSID, a Bonjour name, or identical text such as "Iris Launch."

4.2. DNS-SD records and names

The proposed private service type is `_rt-gs._tcp.local.`; this document does not claim an IANA service-name registration. An example instance is `rt-gs1-telem._rt-gs._tcp.local.`. PTR browsing finds instances; SRV supplies the HTTP host/port; TXT supplies compact hints. These are standard DNS-SD mechanisms [1]. The app SHALL publish after its API is listening and withdraw the service on clean shutdown.

```
Service type: _rt-gs._tcp.local.
Instance:     rt-gs1-telem
SRV target:   rt-gs1-telem.local.
SRV port:     8766
TXT: p=1; node=<UUID>; station=gs1; role=telem; api=/api/v1
```

Each TXT key/value is a separate DNS TXT string; the semicolon layout above is display notation only. Total TXT payload SHALL be at most 512 bytes in this profile. No credentials, telemetry samples or rapidly changing status belong there. The HTTP identity response is authoritative; TXT is an untrusted discovery hint. A disagreement SHALL block normal attachment until rediscovery resolves it.

Preferred hostnames follow the user's `rt-base-*/rt-gsN-*` convention. If an application cannot safely publish that hostname, SRV MAY target the machine's existing resolvable hostname or app-owned mDNS host name. Clients SHALL use SRV/A/AAAA resolution and SHALL not construct a URL by appending `.local` to the service instance. Global OS hostname changes are unnecessary.

mDNS operates on the local link using UDP 5353; cached records are not heartbeats [2]. Access points must permit peer traffic and multicast. Routed VLANs require an explicitly configured discovery gateway or manual endpoints. The protocol SHALL support direct unicast IPv4 and IPv6 endpoints; link-local IPv6 addresses require the local interface scope. URLs use bracketed IPv6 literals.

4.3. Ports and compatibility

Function	Proposed default	Rule
mDNS	UDP 5353	Managed by the discovery implementation.
HTTP + WebSocket	TCP 8766	Configurable; advertise the actual bound port. Do not silently bind a different service at the requested port.
SRT video sessions	UDP 50000–50063	Configurable listener allocation range on video nodes; return actual session endpoints. Client source ports are OS-assigned.
Old snapshot link	TCP 8765	Existing v0a protocol is distinct. No wire compatibility or automatic bridge is implied.

DISC-01 Automatic discovery SHALL fall back to an operator-entered HTTP endpoint. Manual endpoints undergo the same identity/mission/version checks. An IP change SHALL trigger re-resolution and a new handshake, not a new logical station. Duplicate appearances of the same node SHALL merge only when boot and identity agree.

DISC-02 Major-version mismatch SHALL be incompatible. For major 1, unknown optional fields SHALL be ignored and missing required fields rejected. New required behavior needs capability negotiation or a new major version. An unsupported event kind SHALL be logged and ignored without losing its cursor.

5. HTTP API and common encoding

The normative monitoring baseline uses HTTP/1.1 and WebSocket on a team-controlled LAN. It provides no cryptographic peer authentication or confidentiality. A node UUID and discovery name are identifiers, not proof of trust. Network publication SHALL be visible to the operator. Expose no arbitrary files or shell execution. HTTPS/WSS with provisioned identities is a future compatible transport profile; it is required to be designed before enabling remote hardware commands on an untrusted network. This draft's network interface cannot actuate flight hardware.

All JSON uses UTF-8 and `application/json`; no NaN or infinity is legal. Unavailable measurements use `null`, never zero or an empty string. Counters that can grow beyond JavaScript's exact integer range use decimal strings with no leading zeros except `"0"`. UUIDs use canonical lower-case hyphenated form. UTC timestamps use RFC 3339 with `Z`; `null` is allowed when wall-clock time is unavailable. Monotonic milliseconds are decimal strings. Request bodies are limited to 64 KiB; JSON responses and WS messages to 256 KiB. Raw packets within a message are limited to 64 KiB decoded bytes. Larger future payloads require another negotiated profile, not silent truncation.

Method	Path under `/api/v1`	Semantics
GET	`/info`	Identity, capabilities, clocks and protocol version.
GET	`/status`	Atomic current hardware/network state with its event watermark.
GET	`/telemetry/latest`	Latest accepted observation per known vehicle; unknown attribution is a separate entry. Empty array on video-only nodes.
GET	`/streams`	Channel/rendition manifest; empty array on telemetry-only nodes.
GET (WS)	`/events`	WS event subscription; subprotocol `rt-gs.v1`.
POST	`/video/sessions`	Create one authorized-by-profile receive session, detailed in Section 8.
PATCH	`/video/sessions/\{id\}`	Renew that session's lease.
DELETE	`/video/sessions/\{id\}`	Release its resources; repeat is harmless.
POST	`/commands`	Reserved; return 501 `unsupported` in this revision. No dispatch.

GET responses SHALL include `Cache-Control: no-store`. No telemetry endpoint is backed by a constantly rewritten `telem.txt`. Clients SHOULD use a five-second HTTP timeout and cancel obsolete requests after identity changes. The server SHALL reject non-local browser origins by default rather than provide wildcard cross-origin access; this is not a substitute for authentication.

Every successful GET body includes `protocol`, `node_id`, `boot_id`, `mission_id` and `generated_mono_ms`, followed by its endpoint payload. Snapshot endpoints also include `as_of_event_seq`, the atomic journal watermark. A health response is independent of packet freshness. `/info` additionally contains station, role, vehicle mode, app version, source mode, supported event kinds, resource limits and capabilities. Capability Booleans describe actual implementation; a role label is not a capability.

Error bodies contain `error.code`, `error.message`, `error.retryable` and optional `request_id`. HTTP status mapping: 400 malformed JSON; 404 unknown resource; 405 method; 409 identity/mission or idempotency conflict; 410 expired session; 413 size; 415 content type; 422 invalid field or channel; 429 capacity (with `Retry-After`); 501 unsupported operation; 503 device/pipeline unavailable. Errors SHALL not expose secrets or stack traces. GET never changes hardware state.

6. WebSocket lifecycle and event delivery

WebSocket framing, masking, close and ping/pong behavior follow RFC 6455 [3]; use a maintained library. Application messages are one UTF-8 JSON object per WebSocket text message, regardless of underlying frame fragmentation. Binary messages and compression extensions are not used in 1.0. The server SHALL select `rt-gs.v1` during upgrade or reject the upgrade.

6.1. Connection setup and synchronization

1. Client resolves a service, reads `/info`, checks identity, mission, major version and capabilities, then opens `/events`.
2. Server sends `hello` within one second. Its payload contains the same identity, limits, available retained-event range and heartbeat interval. The client SHALL compare boot/node identity with its HTTP result.
3. Client sends `subscribe` within five seconds. Required payload: `client_node_id`, `client_boot_id`, `mission_id`, `accept_source_modes` and `resume`. The last field is `null` or an object containing the server's previous `boot_id` and `after_event_seq`. Default accepted mode list is `["live"]`.
4. Server rejects an identity/mission/mode mismatch with an `error` message then closes 1008. Otherwise it sends `ready` describing `sync_mode`, a captured barrier `through_event_seq`, and the retained range. It atomically captures the barrier and registers subsequent live events.
5. For a valid retained resume cursor, deliver events strictly after the cursor through the barrier. With no cursor, send `snapshot`; with an expired cursor or changed boot, send `reset` followed by `snapshot`. A snapshot is atomic at the barrier and includes status, latest observations and streams. It does not recreate the lost event history.
6. Send `sync.complete` at that same barrier, then queued/live events in ascending event order. A display can show the snapshot immediately, but SHALL not call synchronization complete before this message.

Protocol 1.0 subscribes to all supported events on that node, not filtered topics. Consumers may ignore unused event payloads but SHALL advance the event cursor. This keeps sequence gaps unambiguous. Initial synchronization SHALL complete within five seconds or disconnect with 1013. Heartbeats continue during replay. If a replay cannot fit the per-client limits, issue an explicit reset and fresh snapshot at a new barrier; do not loop indefinitely replaying the same backlog.

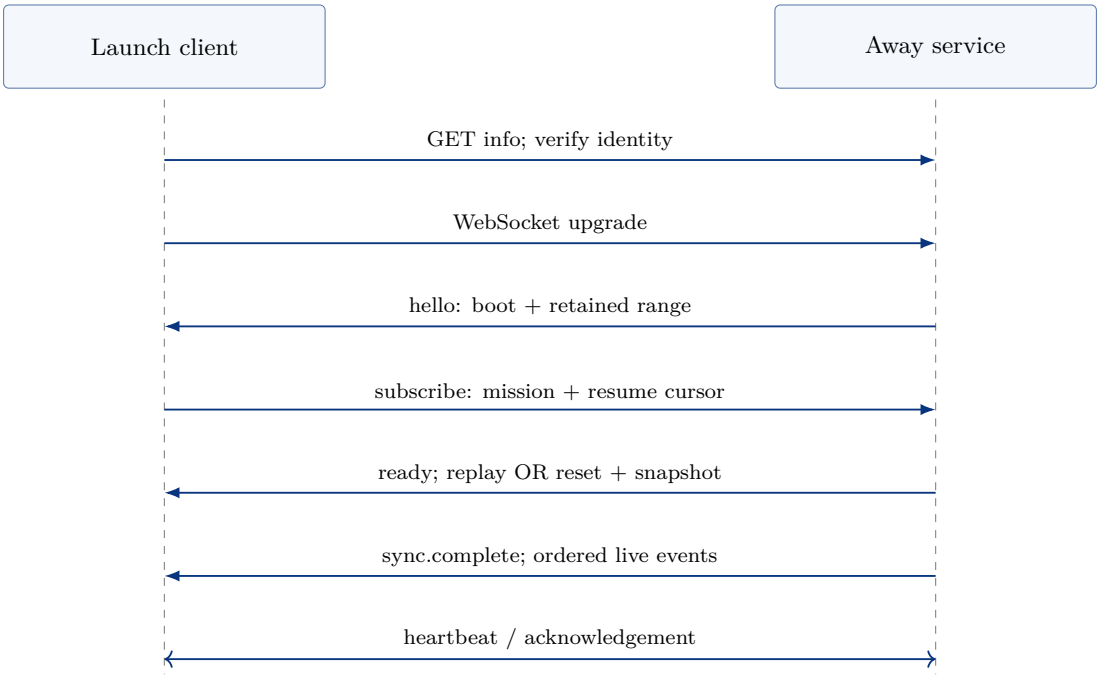


Figure 2: Synchronization has an atomic event barrier. A snapshot repairs current state; a reset explicitly records unavailable history.

6.2. Message envelope and journal

Envelope field	Normative meaning
`v`	Integer 1; major wire version.
`kind`	One of the message kinds below, or a future optional extension.
`node_id`, `boot_id`	Identity of the message origin, not its network relay.
`mission_id`	Mission UUID or null before association.
`source_mode`	`live`, `replay` or `simulation`.
`event_seq`	Decimal-string journal sequence for retained events; null for control/heartbeat messages. Starts at 1 per boot, strictly increasing by one.
`sent_mono_ms`	Milliseconds since origin service start, monotonic, decimal string. Replays carry original event timing in payload and current send time here.
`sent_time_utc`	UTC send time or null; never used alone to establish freshness.
`payload`	Object defined by the message kind; unknown optional fields are ignored.

Kind	Retained?	Payload contract
`hello`	No	Identity, retained range, heartbeat and limits.
`subscribe`	Client	Client identity, mission, accepted source modes, resume cursor. Client control messages use the same envelope with null event sequence.
`ready`, `sync.complete`	No	Synchronization mode and barrier; completion repeats that barrier.
`reset`	No	Reason (`new_boot`, `cursor_expired`, `overflow`), old cursor and new barrier. Marks discontinuity in the receiver archive.
`snapshot`	No	Barrier and complete current status/latest/streams state.
`heartbeat`	No	Heartbeat counter, source monotonic send time, current event head, boot identity and latest source-age summaries.
`heartbeat.ack`	Client	Echo heartbeat counter; server can measure round-trip time without clock synchronization.
`telemetry.observation`	Yes	Received packet, decoded values and provenance (Section 7).
`status.changed`	Yes	Full replacement status object, not a partial patch.
`streams.changed`	Yes	Full replacement manifest after camera, rendition or media-epoch changes.

Kind	Retained?	Payload contract
`error`	No	Error code, human-readable message, retryable flag and relevant request/cursor.

The journal retains up to 30 seconds or 16 MiB of serialized retained messages, whichever limit is reached first. It SHALL advertise the actual first/head sequence; clients SHALL not assume a full 30-second window. Recording archives are distinct and can retain more history. Delivery is at least once across reconnects; consumers deduplicate by `(node_id, boot_id, event_seq)`. Repeated sequences with different payloads are a protocol error.

The per-client send queue is limited to 2 MiB and two seconds of oldest pending message age. Exceeding either limit disconnects that client with 1013; the serial worker and journal continue. If possible send an error before closure, but the client SHALL infer a discontinuity from its next cursor negotiation even when the socket dies first. Server implementations SHALL not drop telemetry events silently to protect a slow screen. UI rendering may coalesce its own copies.

Malformed UTF-8 closes 1007, excess size closes 1009, and invalid required fields close 1008. A normal shutdown closes 1000. Receivers SHALL log the reason and last accepted cursor. Backoff after transport failure is 0.5, 1, 2, 4, then 8 seconds, each with uniform ±20% jitter; reset after 30 seconds of healthy operation. Conflict, mission mismatch and incompatibility require changed configuration or new discovery evidence rather than continuous retry.

6.3. Liveness, clock handling and freshness

Server application heartbeats occur every 1 second; the client acknowledges them. Standard WS ping/pong may additionally detect socket failure but does not replace application heartbeat. A client measures age using its own monotonic clock: below 3 seconds is ONLINE, 3 to below 10 is STALE, and at 10 seconds is OFFLINE. A definite socket closure is immediately DISCONNECTED. A new heartbeat can restore ONLINE, but each board/RF/feed status retains its own age and state. These thresholds are proposed defaults and SHALL be reported in the UI settings.

Monotonic values from different computers SHALL never be subtracted. An observation contains source reception time and source send time in the same boot clock. Source age at send is their difference; the receiver can then add local time since arrival. That value is a lower-bound age, since one-way network delay is not measured. Label it accordingly when clocks are unsynchronized. RTT/2 is not an exact one-way delay. Cross-station time alignment requires clock offset and uncertainty from an actual clock-synchronization scheme, with quality `unknown`, `estimated` or `synchronized`. UTC presence alone does not establish synchronization. Unknown uncertainty is `null`.

Default RF telemetry stale threshold is 2 seconds and lost threshold is 10 seconds after source reception; deployments with intentionally slower packet rates SHALL configure and publish appropriate values. A new network heartbeat SHALL not reset packet age. Original device time and boot identity are retained when available; ground monotonic time is not a rocket timestamp.

7. Telemetry observation and status contracts

7.1. Packet provenance and vehicle attribution

Each `telemetry.observation` payload SHALL contain these objects:

Object	Required contents and interpretation
`observation_id`	Origin node UUID, boot UUID and a decimal observation counter, joined with colons. Unique even when packet bytes repeat.
`receiver`	Station, logical board ID and a `connection_id` UUID that changes on each serial reconnect. RSSI/RX-RSSI in dBm or null; observed decoder gaps and reject counts are receiver statistics.
`vehicle`	ID: `balius`, `sustainer`, `booster` or `unknown`; basis: `packet_id`, `confirmed_switch`, `operator_assignment` or `unknown`; target epoch UUID or null.
`packet`	Serial protocol name/version, decoder version, packet type, original sequence and device boot/time when available, `raw_b64`, `raw_sha256`. Raw bytes exclude ground-network framing.

Object	Required contents and interpretation
`timing`	Source `received_mono_ms`, `received_time_utc`, wall-clock quality and uncertainty; original device time is retained separately in packet metadata.
`values`	Canonical measurements below, plus the decoder's complete non-canonical fields under `native`. No legacy fields are intentionally dropped.
`quality`	Validation result, attribution confidence and warnings; unknowns explicitly represented. Only accepted decoded packets enter this event kind.

`raw_b64` uses standard padded Base64; the SHA-256 digest covers the exact decoded byte sequence and is lower-case hexadecimal. Both may be null only when the upstream source cannot supply bytes, with a reason in quality. The baseline serial ingestion SHALL preserve bytes. Bad-checksum/malformed packets are counted and optionally retained in a diagnostic archive; they SHALL not become accepted telemetry merely because the network transports them successfully.

TEL-01 Iris observations attributed only by desired switch position SHALL remain `vehicle.id=unknown`. A static operator assignment MAY label a source, but its basis remains visible and SHALL not count as hardware confirmation. Confirmed switch changes create a new target epoch and a transition interval in which delayed buffered packets remain unknown unless packet-level identity resolves them. The firmware contract must define that interval before use.

Packet deduplication is distinct from event deduplication. When the rocket has a reliable vehicle/boot/packet sequence, use that identity plus payload consistency to group multiple receptions. Otherwise identical raw hashes, packet type and bounded time proximity provide only a duplicate candidate. Repeated identical packets at different times are possible. Keep every receiver observation; do not merge unknown vehicle/boot sequences or average duplicate rocket measurements.

7.2. Canonical values and physical frames

Field under `values`	Representation and units
`state`	Object with original numeric `code` (or null) and decoded `name` (or null); retain unrecognized original codes.
`position`	`lat_deg`, `lon_deg` on WGS84; `alt_m`, `alt_datum`, `fix`. Datum: `wgs84_ellipsoid`, `msl`, `agl_launch` or `unknown`. Missing fix never becomes $(0, 0)$.
`baro_altitude`	Value in metres and explicit datum; launch-relative is not MSL.
`velocity_enu_m_s`	Three-element east/north/up vector only if available in that frame; otherwise null. A scalar integrated velocity is a separate native field.
`orientation`	Null or body-to-ENU quaternion `quaternion_wxyz`, right handed, scalar first, nose body $+Z$, explicit body-axis definition. Unit norm within 0.001 before normalization.
`flight_status`	`motor_ignited`, `motor_burning`, `parachute_deployed`: true, false or null independently, with evidence source. Do not infer confirmation from flight phase.
`native`	Complete decoder-specific JSON object, with `native_schema` identifier. Documents units, meanings and validity under that decoder version.

The existing Zephyrus roll/pitch/yaw integrals are not Euler attitude. They remain native values; they SHALL not populate orientation. Three-axis rendering requires a validated orientation source. Future estimator output SHALL use a distinct source/type with input observation IDs, frame, time, uncertainty and validity; this protocol does not select or qualify a sensor-fusion algorithm.

7.3. Status and stream health

The status object contains `application`, `telemetry_board`, `pointer`, `video_inputs`, `network` and `clock` objects. Inapplicable hardware is `null`, not disconnected. Each hardware object includes availability (`unsupported`, `disconnected`, `connected`, `error`), a change timestamp and a human-readable reason. Telemetry adds last observation identity/age, validity counts, RF metrics and desired/confirmed target and uplink enable states. Pointer adds mode, commanded azimuth/elevation and nullable measured pose/acknowledgement. Commanded azimuth is degrees clockwise from true north in $[0, 360)$; elevation is above the local horizon, in degrees. If calibration to true

north is unavailable, report that fact and do not pretend the encoder reference is true north. Physical travel limits are separate.

Each video input reports capture connection, last source-frame age, source resolution/FPS, pipeline error and nullable RF lock/quality with its evidence. Network status reports interface and observed endpoint information; LTU signal metrics remain null until a documented adapter supplies them. Complete status is sent on meaningful change and at most twice per second; heartbeat age summaries update every second. Store original change times, so retransmitting a status does not make stale hardware look freshly measured.

8. H.264 media profile and SRT transport

8.1. Codec, container and transport are separate choices

VID-01 Protocol 1.0 SHALL carry H.264/AVC video in MPEG-TS over SRT live mode. HTTP advertises and allocates streams; WebSocket carries status. Video bytes SHALL not share the telemetry WebSocket. SRT is the proposed transport binding; H.264 was the explicitly accepted codec choice.

The video node is the SRT *listener and sender*; Launch Video is the SRT *caller and receiver*. Caller/listener describes who establishes the connection, not who produces the video. Connections are unicast on the LAN. No Internet rendezvous service, browser or external media server is needed. SRT's receive buffer trades delay for recovery of late/retransmitted packets; its latency setting is not the complete camera-to-screen latency [4].

Item	Required baseline
H.264 decoder profile	Constrained Baseline, level at most 3.1, progressive 8-bit 4:2:0. No B-frames. Other profiles require an explicitly advertised extension.
Random access	IDR at least once per second; repeat SPS/PPS at every IDR. New subscribers begin with parameter sets and a decodable IDR.
MPEG-TS	One program with one H.264 video elementary stream (stream type 0x1B); Annex-B NAL representation, PTS and PCR. Audio absent in 1.0. Receivers use PAT/PMT rather than hard-coded PIDs.
Tables and timing	PAT/PMT at startup and at least every 100 ms; PCR interval at most 40 ms. PTS uses the 90 kHz timebase; handle its 33-bit wrap. These are selected application-profile limits.
SRT mode	Live, caller/listener, timestamp-based packet delivery, too-late packet drop enabled. Baseline receiver latency 120 ms; the session reports actual negotiated value.
Payload	Up to 1316 bytes (seven 188-byte TS packets) per SRT application send; shorter final groups are permitted. Do not append a custom JSON header to media.
Connect/read behavior	Connect timeout 3 s; no decoded frame for 2 s is stale, for 5 s unavailable. An explicit closed connection is immediately unavailable. Reconnect uses the session procedure below.
Resource boundaries	One capture per device; one encoder per active channel/rendition; fan out encoded data to distinct per-viewer SRT sessions. A slow viewer cannot block other viewers.

MPEG-TS parameters and SRT option mappings must be checked against the packaged media library. FFmpeg documents these muxer and protocol options [7, 6]; using another compliant library is allowed. In particular, an adapter must convert this document's millisecond values to the units required by that API. FFmpeg SRT latency options use microseconds.

8.2. Renditions and manifests

Rendition	Image/FPS ceiling	Proposed video bitrate	Intended use
`preview`	Fit within 320 × 180; 10 fps	250 kbit/s target; 300 kbit/s cap	Eight small remote views
`main`	Fit within 1280 × 720; 30 fps	3 Mbit/s target; 4 Mbit/s cap	Selected large channel

Preserve aspect ratio with even dimensions; do not distort a 4:3 analog capture. Do not invent frames to claim a source FPS it does not deliver. Encoder output and source FPS SHALL both be reported. These are network

rendition ceilings, not assumptions about the capture device's native resolution. A profile may deliver lower quality when declared in its actual encoding parameters. Full-HD, audio and additional codecs are extensions, not baseline requirements.

`/streams` returns `streams`, an array of entries with: `stream_id`, `channel_id`, `station`, `vehicle`, `radio_type`, `source_mode`, `capture_state`, `rf_lock`, `media_epoch`, and `renditions`. A stream ID is `station.channel_id`, for example `gs4.booster.analog`; globally identify a publisher with node/boot as well. Each rendition describes name, codec, container, transport, actual resolution/FPS, target/cap bitrate and `available` plus reason. A configured disconnected camera remains in the manifest with unavailable renditions. No hardware camera identifier is needed by remote clients.

`media_epoch` is a UUID for the channel's current capture/encoding timeline. It changes on camera replacement, capture restart, or encoder/timestamp reset. Starting a previously unused rendition alone may join the existing timeline; any change that invalidates its timing requires a new channel epoch and restarts all sessions for that channel. The service boot and media epoch are independent. Send `streams.changed` before issuing new sessions for a changed epoch.

8.3. Session allocation, connection and lease

The client discovers metadata first; it does not guess a media port. To receive one rendition it POSTs the following body to `/video/sessions`:

```
{
  "request_id": "<UUID>", "client_node_id": "<UUID>",
  "client_boot_id": "<UUID>", "mission_id": "<UUID>",
  "expected_server_boot_id": "<UUID>",
  "stream_id": "gs1.sustainer.digital",
  "rendition": "main", "expected_media_epoch": "<UUID>"
}
```

`request_id` makes session creation idempotent: within a service boot, repeating the same ID and identical fields returns the original session result. Different fields return 409. Keep that record until at least 60 seconds after the session ends; an expired recorded result returns 410, never a fresh session. Clients SHALL retry an uncertain POST only within this interval with the same ID; afterward perform cleanup/reconciliation and create a new explicit request. For the owning client, `/streams` MAY include its active session IDs to aid reconciliation; this optional convenience is not needed for baseline operation.

The service validates mission, boot, channel assignment, source mode, media epoch, capacity and camera state; starts/reuses the encoder; binds one available UDP listener in its configured pool; then returns HTTP 201. Do not return success before a listener is ready. The response includes `session_id`, `session_token`, `server_boot_id`, `media_epoch`, `stream_id`, `rendition`, `transport`, `lease_ms=30000` and actual encoding parameters. `transport` contains resolvable host, UDP port, `mode=caller` (for the client), `srt_streamid`, `latency_ms=120`, `container=mpegts` and `codec=h264`. Use the selected reachable interface, never `0.0.0.0` as the advertised destination.

The SRT Stream ID is the ASCII string `rt1:<session_id>:<session_token>`. The token is 128 bits of cryptographically random data encoded as 32 lower-case hex digits. The caller sends it at connection setup; the listener compares it exactly before accepting one receiver. Stream ID is connection metadata available to the listener [5]. This token prevents accidental session mixups; on this plaintext LAN profile it does not provide encryption or strong peer trust. Do not include it in logs, DNS records or screenshots.

Each allocated listener serves exactly one client session; additional connection attempts are rejected. Sharing one source therefore means sharing encoded bytes across independent SRT sockets, not assuming a single socket automatically broadcasts. Baseline capacity is four active sessions per video node, with at most two main and two preview sessions. Unavailable capacity returns 429 and a one-second retry hint. These limits support the current video wall, including two promoted channels at the same Away station.

The client renews every 10 seconds with PATCH, body `\{"lease_ms":30000\}`, and header `Authorization: Bearer <session_token>`. Only 30000 is accepted in 1.0. Expiry uses the server's monotonic clock, extended to 30 seconds from successful renewal. PATCH success returns the remaining lease; no cross-computer comparison of absolute expiry timestamps is required. Expiry or DELETE closes that session, returns its UDP port and stops unused encoding after a two-second grace period. DELETE uses the same header and returns 204 even after prior deletion; unknown tokens cannot affect another live session. Invalid tokens return 403.

On media loss, DELETE/release the previous session and allocate a new one with a new request ID after rechecking boot/epoch. Do not reuse an accepted SRT session token for a second socket. A server boot change invalidates all

leases and tokens. Loss of the status WebSocket alone need not destroy healthy media immediately; mark status stale and retain media only while its own lease/frames remain valid.

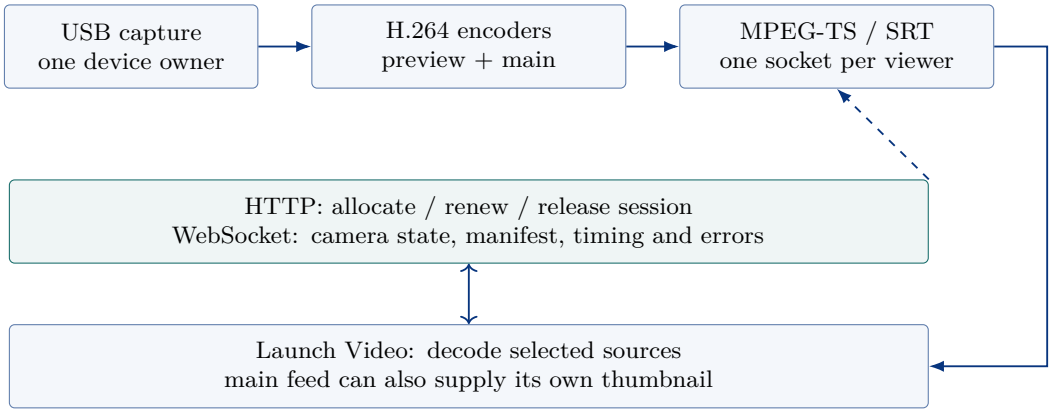


Figure 3: Video pixels and control/status travel separately. An HTTP session establishes a specific channel, rendition and media epoch before SRT begins.

8.4. Promotion, bandwidth and failure recovery

VID-02 Launch Video SHALL normally request previews for all eight Away feeds. Promoting a feed requests its main rendition, retains the preview until the first main IDR is decoded, then uses that same main decode for its thumbnail and releases the now-redundant preview session. Demotion performs the reverse.

VID-03 No main rendition SHALL be requested for a logical channel that the publishing station does not possess. A source swap SHALL update the label and epoch before displaying new pixels. A failed swap SHALL leave the previous view intact with an explicit error; it SHALL not mislabel old pixels.

A conservative budget for eight previews at the cap and three main feeds is $8(0.3) + 3(4) = 14.4$ Mbit/s of encoded video. Adding a provisional 30% allowance for encapsulation/retransmission gives 18.72 Mbit/s. This deliberately counts previews also shown from main and so overestimates steady-state use. It does not include Wi-Fi airtime contention, recording transfer, other users or unusually high loss. It is not a claim of LTU throughput. Measure every Away path and the shared Launch link under concurrent load; provision at least twice this planned load as measured sustained application capacity for the initial deployment.

If capacity falls, reduce preview rate/bitrate first, then offer reduced main quality explicitly. Prioritize serial ingestion, event publication and heartbeat processing over video work. Separate sockets alone cannot guarantee QoS on a saturated radio; encoder caps, admission limits and network configuration are part of acceptance testing. Initial implementation MAY use fixed profiles with operator quality selection; automatic congestion adaptation is not assumed to be supplied by SRT.

8.5. Media timing and recording

The receiver SHALL use decoder PTS for playback and its own monotonic arrival clock for frame-staleness display. Capture time means receipt of a USB frame by the publishing host, not the rocket camera's exposure time. The receiver/RF path may add unknown latency before USB capture. SRT timestamps do not provide rocket camera timestamps.

For correlation, each active rendition's status SHALL report `media_epoch`, `pts_90k` as a decimal unwrapped counter, `capture_mono_ms`, `capture_time_utc`, and clock uncertainty for one identified encoded frame at least once per second. TS PTS is that counter modulo 2^{33}. Two rendition encoders SHALL derive times from the same capture timeline; frames may be omitted for preview. Resetting the timeline changes the epoch. This mapping supports approximate telemetry/video correlation; accurate cross-station overlays remain unqualified until clock and capture delays are measured.

Local recording SHALL be independent of remote viewer quality. When the operator enables recording, keep local video/telemetry with source IDs, mission, epochs, timestamps, clock quality, decoder/codec settings and gap events. Launch recording can archive what it receives but SHALL not claim to contain frames lost upstream. Disk-full or recording failure SHALL be visible without blocking live acquisition. After a video gap, resume on an IDR and reset the decoder as needed; do not display corrupted inter-frame predictions as a current reliable image.

Part III.
Verification and implementation boundaries

9. Performance targets and conformance checks

Targets, not measurements No network or media implementation was built or benchmarked for this document. The following figures are acceptance targets for the proposed design and must be validated on the production machines.

Quantity	Proposed acceptance target	Measurement boundary
Discovery and attachment	Within 5 s after service publication on a healthy LAN	Publication to valid identity shown, using a fresh browse.
Telemetry forwarding	p95 at most 100 ms; p99 at most 250 ms	Accepted serial packet at publisher to event delivery at Launch, with qualified synchronized clocks or a loopback measurement. Excludes RF delay.
UI update	At least 10 Hz for changing displayed values	Render coalescing permitted; archive and event history must preserve full input.
Main video latency	p95 at most 500 ms	USB frame acquisition at Away host to Launch presentation; camera/RF delay measured separately.
Thumbnail promotion	First main frame within 2 s	User selection to first correctly labeled main frame on a healthy LAN.
Health indication	Stale at 3 s; offline at 10 s	Since last valid app heartbeat; closure is immediate disconnect.
Video freshness	Stale at 2 s; unavailable at 5 s	Since last decoded new frame, using receiver-local monotonic time.
Endurance	2 h with ten nodes, eight previews and up to three mains	No unbounded queues/memory; no unreported event loss; UI remains responsive.
Telemetry capacity fixture	100 observations/s at 2 KiB per telemetry node	Five publishers; sustained aggregate roughly 1 MiB/s before framing. Replace with measured actual worst-case rates if higher.

The fixture rate is a test workload, not a claim about Zephyrus transmission rate. Bandwidth calculations in the video section exclude this telemetry workload. Use synthetic packets for network stress and separate recorded real packets for decoder fidelity. Measure camera-to-screen delay with a timed visual stimulus or frame-tagged capture; do not substitute ping latency or configured SRT latency. Publish machine specs, network topology, encoding settings, clock uncertainty, percentiles and errors with results.

Test	Requirement coverage	Required evidence / pass condition
T01	SYS-01, DISC-01	Start all ten roles in mixed Mac/Windows deployment. All slots resolve once; DHCP address change and manual endpoint reconnect preserve node identity.
T02	ID-01, ID-02	Launch duplicate slot and cloned node IDs. Show conflict; no automatic source takeover. Changing role/mission creates a new boot and closes old sessions.
T03	DISC-02	Reject major mismatch and malformed identity. Accept an unknown optional field/event while retaining cursor continuity.
T04	UI-01–04	Unplug serial while network remains online; stop RF packets while USB remains open; feed a receiver no-signal image. Correct independent states and unknown RF lock.
T05	SYS-02, PKG-01	Duplicate camera/serial selections stay prohibited. Two local views and remote viewers use one capture owner; no extra external runtime installation.
T06	SYS-03	Close either Launch app and interrupt one Away path. Other peers, local recording and the remaining Launch app keep working.
T07	SYS-04, UI-05	Balius and Iris matrices exactly match station assignments. Eight thumbnails; promotion only to a matching channel; local digital remains available.

Test	Requirement coverage	Required evidence / pass condition
T08	SYS-05, TEL-01	Unconfirmed Iris switch never labels a packet as confirmed. Test delayed packets after confirmed switch/serial reconnect and preserve target epochs.
T09	DATA-01–02	Preserve raw bytes/digest, full native fields and receiver RSSI from GS1/GS2/GS3 recordings. Same packet at three sites becomes three observations and one candidate rocket measurement.
T10	DATA-03	Inject replay and simulation advertisements into a live mission. No silent mixing; identity/mode mismatch is visible.
T11	NET-02	Disconnect inside and outside retention window, then restart publisher. Resume without duplicate processing; reset/snapshot declares gaps and new boot.
T12	NET-01–02	Stall a subscriber; exceed size/age limits. Only that client is disconnected; journal/serial ingestion continue. No silent event loss.
T13	UI-02, DATA-01	Skew wall clock by 5 min and step it backward; disconnect clock sync. Local freshness still works; interstation time precision becomes unknown.
T14	VID-01, PKG-02	Inspect codec/profile, SPS/PPS/IDR cadence, TS tables and timestamps. Decode each supported Mac/Windows sender/receiver pairing; verify packaged SRT and fallback encoder.
T15	VID-02–03	Repeatedly promote/demote thumbnails at capacity. Main decode supplies thumbnail; unused session/encoder resources are released; labels never mismatch pixels.
T16	UI-06, VID-03	Lose a selected feed or reject main quality. Keep source explicit, show stale overlay or same-source preview; no hidden source substitution.
T17	Session lifecycle	Retry uncertain POST with same ID; altered body returns 409. Expire lease, send wrong Stream ID/token, restart source, change camera epoch. No cross-session video.
T18	NET-01, VID-01	With eight previews and three mains, introduce 1% random packet loss and 50 ms variable delay, then a 2 s link outage. Record media degradation, telemetry latency and recovery; no growing playback backlog.
T19	NET-01–02	Run capacity fixture and two-hour endurance test. Log queue high-water marks, CPU/GPU load, RSS and bandwidth; investigate all loss/reset events.
T20	UI-07, DISC-01	Block mDNS while permitting unicast, deny local-network permission, enable AP isolation. Distinguish causes; manual endpoint works only when unicast is reachable.
T21	DATA-01, UI-04	Absent orientation/motor/chute/pose feedback stays unknown. No phase-based attitude, RF lock or measured-pointer fabrication.
T22	Archive contract	Disk-full and recorder failure do not stall ingestion; session manifests capture frame gaps, epochs and source IDs.
T23	Hardware boundary	Send `/commands` requests including switches/uplink/pointer actions. Receive 501; observe no serial writes or claimed board acknowledgement.
T24	HTTP/WS limits	Test invalid UTF-8, JSON, oversized payloads, non-finite numbers, unknown resources and counter conflicts; expected errors and bounded resource use.

10. Implementation order and open decisions

Implement identity/discovery and status first; then loss-aware telemetry transport and archival provenance; then one H.264/SRT channel across Mac/Windows; then the full video wall, previews, session limits and fault injection. Finally qualify the actual radios and cameras with all ten machines. Each stage preserves the existing local device-control boundaries. The proposed protocol is independent of whether Python bindings, a bundled native helper or another library implements the transport.

Open item	Needed decision/evidence	Behavior until resolved
Telemetry switch	Firmware command, acknowledgement, vehicle attribution and buffer transition semantics	Desired/confirmed separated; unknown target where unverified.
Uplink enable	Launch board API, enable confirmation and power-up/reconnect state	Network capability false; unsupported; no invented serial bytes.

Open item	Needed decision/evidence	Behavior until resolved
Remote pointer control	Ownership, authorization, command/feedback schema, limits and recovery semantics	Monitoring only; local owner retains control. A selected route is descriptive.
LTU metrics	Model/firmware management API and deployment access	Metrics unknown; IP/heartbeat health still available.
Timing/fusion	Clock service, acceptable uncertainty, rocket boot/sequence identity and estimator model	Preserve raw observations; no claimed fused navigation estimate.
Capture hardware	Actual resolution, FPS, compressed USB formats and encoder availability	Negotiate qualified baseline; report actual format and errors.
Field network	AP/bridge topology, multicast behavior, measured throughput and interference	Manual endpoints; conservative bitrate caps; no claimed performance.
Security extension	Peer provisioning, TLS trust, command permissions and audit requirements	Controlled-LAN monitoring profile only; all hardware commands unsupported.

Future hardware commands require a separate revision defining stable request IDs, expiry, authority, requested/dispatched/confirmed/failed/unknown outcomes and device acknowledgement. A TCP write or HTTP success is not a hardware outcome. Non-idempotent actuation SHALL not be retried merely because acknowledgement was lost. These constraints reserve the boundary; they do not invent the unresolved board protocol or enable remote actuation in this draft.

11. Traceability and document maintenance

SYS/UI/DATA/NET/PKG/ID/DISC/TEL/VID identifiers are stable requirements. T01–T24 are future acceptance cases. Changes to mandatory wire fields, event meaning, channel mapping or transport behavior require a revision entry and example updates. Keep design version, wire protocol version and application version separate. Application support is established by implementation and test evidence, not by this document's existence.

The accompanying JSON files are protocol examples, not real flight measurements or cryptographic credentials. A documentation validator checks their syntax, cross-file identities, observation hash and referenced paths; it does not certify a server implementation. Rebuild the PDF with `make`. Use `make clean` to remove intermediate TeX artifacts while preserving the final PDF.

A. Example exchanges

These complete JSON files illustrate the normative field names. UUIDs, addresses, tokens and measurements are synthetic. `raw_b64` in the observation example encodes a documentation fixture, not a valid Zephyrus packet; its digest is real for those fixture bytes. The example decoder and native schema names are not claims about production firmware. Token values must be generated at runtime.

A.1. Telemetry service identity


```
{
  "protocol": {"major": 1, "minor": 0},
  "node_id": "11111111-1111-4111-8111-111111111111",
  "boot_id": "22222222-2222-4222-8222-222222222222",
  "mission_id": "33333333-3333-4333-8333-333333333333",
  "generated_mono_ms": "149000",
  "station": "gs1",
  "role": "telem",
  "vehicle_mode": "iris",
  "source_mode": "live",
  "app_version": "future-implementation",
  "supported_event_kinds": ["telemetry.observation", "status.changed", "streams.changed"],
  "clock": {"quality": "unknown", "uncertainty_ms": null},
  "capabilities": {
    "telemetry": true,
    "pointer_status": true,
    "video": false,
    "remote_commands": false,
    "uplink_enable": false,
    "target_switch": false
  },
  "limits": {
    "message_bytes": 262144,
    "request_bytes": 65536,
    "journal_bytes": 16777216,
    "journal_ms": 30000,
    "heartbeat_ms": 1000
  }
}
```


A.2. Client subscription

This client has previously accepted event 40 from the indicated service boot. The server may replay 41 onward only if that cursor remains in its retained range.


```
{
  "v": 1,
  "kind": "subscribe",
  "node_id": "44444444-4444-4444-8444-444444444444",
  "boot_id": "55555555-5555-4555-8555-555555555555",
  "mission_id": "33333333-3333-4333-8333-333333333333",
  "source_mode": "live",
  "event_seq": null,
  "sent_mono_ms": "90000",
  "sent_time_utc": "2026-10-25T18:00:00.000Z",
  "payload": {
    "client_node_id": "44444444-4444-4444-8444-444444444444",
    "client_boot_id": "55555555-5555-4555-8555-555555555555",
    "mission_id": "33333333-3333-4333-8333-333333333333",
    "accept_source_modes": ["live"],
    "resume": {"boot_id": "22222222-2222-4222-8222-222222222222", "after_event_seq": "40"}
  }
}
```

A.3. Received observation

The source has a known Sustainer attribution from a hypothetical packet identifier. Real Zephyrus packets without such identity must use the actual supported basis. Orientation and motor/recovery state remain unknown.


```
{
  "v": 1,
  "kind": "telemetry.observation",
  "node_id": "11111111-1111-4111-8111-111111111111",
  "boot_id": "22222222-2222-4222-8222-222222222222",
  "mission_id": "33333333-3333-4333-8333-333333333333",
  "source_mode": "live",
  "event_seq": "41",
  "sent_mono_ms": "150000",
  "sent_time_utc": "2026-10-25T18:00:00.000Z",
  "payload": {
    "observation_id": "11111111-1111-4111-8111-111111111111:22222222-2222-4222-8222-222222222222:12",
    "receiver": {
      "station": "gs1",
      "board_id": "downlink",
      "connection_id": "aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa",
      "rssi_dbm": -83.0,
      "rx_rssi_dbm": null,
      "decoder_gaps": "0",
      "rejected_packets": "0"
    },
    "vehicle": {"id": "sustainer", "basis": "packet_id", "target_epoch": null},
    "packet": {
      "protocol": "documentation-fixture",
      "protocol_version": "1",
      "decoder_version": "example-1",
      "type": "navigation",
      "sequence": "42",
      "device_boot_id": null,
      "device_time_ms": null,
      "raw_b64": "RE9DLU9OTFktUEFDS0VULTQy",
      "raw_sha256": "db02a19db5954d8274a922dec9abdd3d8581659bbb1852a0efef77a40050205c"
    },
    "timing": {
      "received_mono_ms": "149980",
      "received_time_utc": "2026-10-25T17:59:59.980Z",
      "clock_quality": "unknown",
      "clock_uncertainty_ms": null
    },
    "values": {
      "state": {"code": null, "name": "PREFLIGHT"},
      "position": {"lat_deg": 42.7, "lon_deg": -77.2, "alt_m": null, "alt_datum": "unknown", "fix": true},
      "baro_altitude": {"value_m": 0.0, "datum": "agl_launch"},
      "velocity_enu_m_s": null,
      "orientation": null,
      "flight_status": {
        "motor_ignited": null,
        "motor_burning": null,
        "parachute_deployed": null,
        "evidence_source": null
      },
      "native_schema": "documentation-fixture/1",
      "native": {"temperature_c": 21.5}
    },
    "quality": {
      "validation": "accepted",
      "attribution_confidence": "explicit_id",
      "warnings": ["Documentation fixture; not Zephyrus wire bytes."]
    }
  }
}
```

A.4. Video session request and response

The media publisher is the separate GS1 Video node, not GS1 Telemetry. HTTP 201 returns the already listening endpoint. The token shown here is illustrative and must never be deployed as a constant. `latency_ms` is the requested baseline; actual negotiated latency becomes available after the SRT handshake in session status.

```
{
  "request_id": "99999999-9999-4999-8999-999999999999",
  "client_node_id": "44444444-4444-4444-8444-444444444444",
  "client_boot_id": "55555555-5555-4555-8555-555555555555",
  "mission_id": "33333333-3333-4333-8333-333333333333",
  "expected_server_boot_id": "66666666-6666-4666-8666-666666666666",
  "stream_id": "gs1.sustainer.digital",
  "rendition": "main",
  "expected_media_epoch": "77777777-7777-4777-8777-777777777777"
}
```

```
{
  "session_id": "88888888-8888-4888-8888-888888888888",
  "session_token": "0123456789abcdef0123456789abcdef",
  "server_boot_id": "66666666-6666-4666-8666-666666666666",
  "media_epoch": "77777777-7777-4777-8777-777777777777",
  "stream_id": "gs1.sustainer.digital",
  "rendition": "main",
  "lease_ms": 30000,
  "transport": {
    "host": "rt-gs1-video.local",
    "port": 50000,
    "mode": "caller",
    "srt_streamid": "rt1:88888888-8888-4888-8888-888888888888:0123456789abcdef0123456789abcdef",
    "latency_ms": 120,
    "container": "mpegts",
    "codec": "h264"
  },
  "encoding": {
    "profile": "constrained_baseline",
    "level": "3.1",
    "width": 1280,
    "height": 720,
    "fps": 30,
    "target_bitrate_bps": 3000000,
    "max_bitrate_bps": 4000000
  }
}
```

A.5. Expired resume cursor

The client records a discontinuity, then consumes the following snapshot at the new barrier. It does not fabricate observations 41–89.

```
{
  "v": 1,
  "kind": "reset",
  "node_id": "11111111-1111-4111-8111-111111111111",
  "boot_id": "22222222-2222-4222-8222-222222222222",
  "mission_id": "33333333-3333-4333-8333-333333333333",
  "source_mode": "live",
  "event_seq": null,
  "sent_mono_ms": "190000",
  "sent_time_utc": "2026-10-25T18:00:00.000Z",
  "payload": {
    "reason": "cursor_expired",
    "old_cursor": {"boot_id": "22222222-2222-4222-8222-222222222222", "after_event_seq": "40"},
    "retained_first_event_seq": "90",
    "through_event_seq": "125"
  }
}
```

References

[1] S. Cheshire and M. Krochmal, *DNS-Based Service Discovery*, RFC 6763. https://www.rfc-editor.org/rfc/rfc6763.html.

[2] S. Cheshire and M. Krochmal, *Multicast DNS*, RFC 6762. https://www.rfc-editor.org/rfc/rfc6762.html.

[3] I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455. https://www.rfc-editor.org/rfc/rfc6455.html.

[4] Haivision, *SRT latency and timestamp-based delivery*. https://github.com/Haivision/srt/blob/master/docs/features/latency.md.

[5] Haivision, *SRT API socket options*, including `SRTO_STREAMID`, live delivery and latency options. https://github.com/Haivision/srt/blob/master/docs/API/API-socket-options.md.

[6] FFmpeg, *Protocols documentation*, SRT section. https://ffmpeg.org/ffmpeg-protocols.html#srt.

[7] FFmpeg, *Formats documentation*, MPEG-TS muxer. https://ffmpeg.org/ffmpeg-formats.html#mpegts-1.

[8] MIT Rocket Team, *Rocket GNC Monitor v0a: Operator manual and developer reference*, release 0.1.1, 25 October 2026. Local companion report: v0a manual. Application: https://github.com/MIT-Rocket-Team/RT_Telemetry_UI/tree/v0.1.1.

Standards and library documentation were consulted on 25 October 2026. This report's endpoint names, fields, timers, limits, H.264 encoding profile, session allocation and acceptance thresholds are project design choices; they are not requirements imposed by Bonjour, WebSocket, FFmpeg or SRT themselves.